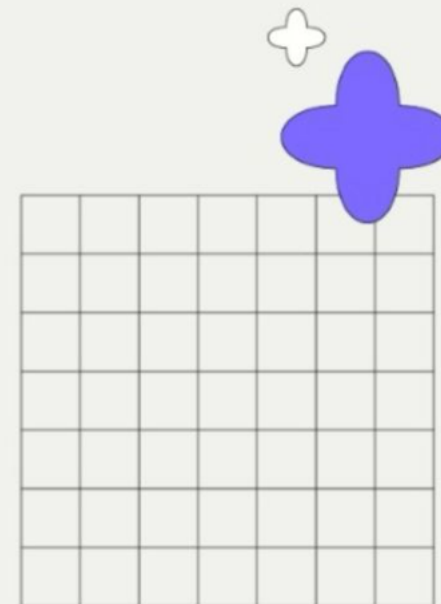
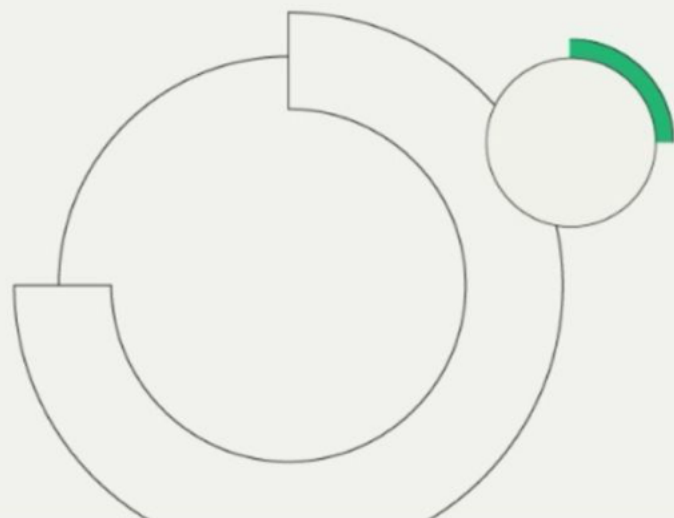
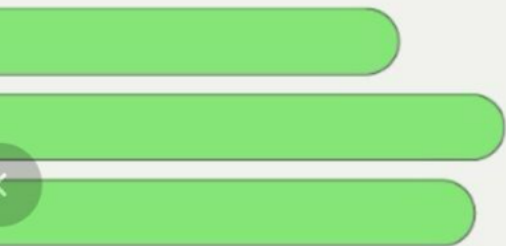
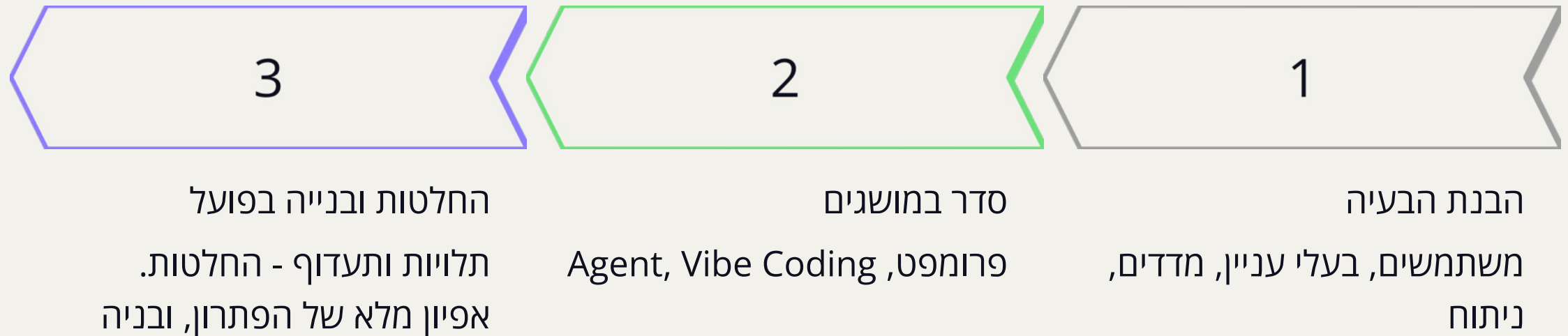


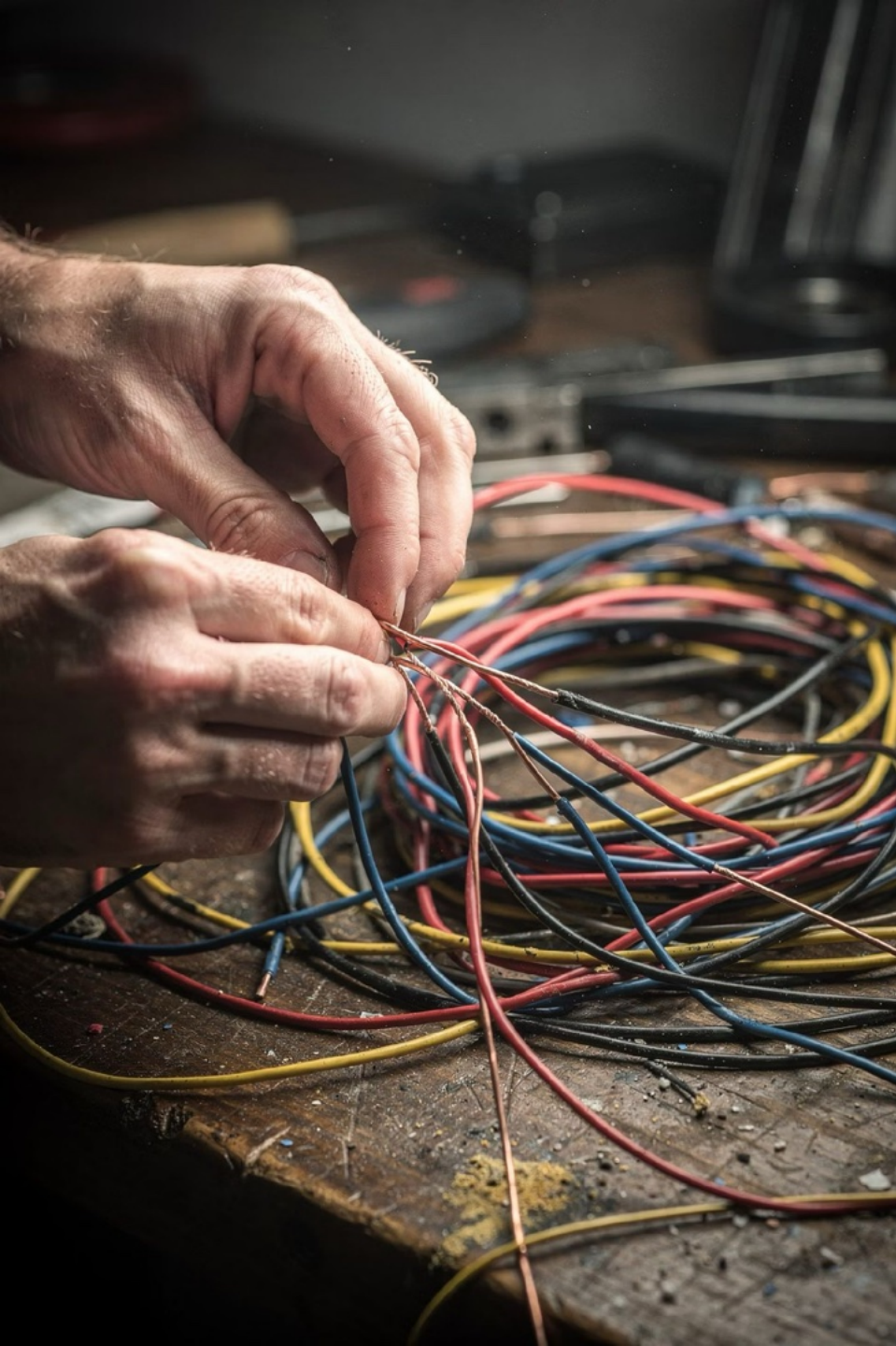


שיעור 7 - תעדוף ותלויות

איפה אנחנו עומדים

עד עכשיו התמקדנו בהבנת הבעיה, המשתמשים, בעלי העניין והמזדים. עכשיו אנחנו מתחילים את תהליך בניית המערכת.





פרומפטים Vibe Coding Agents

שלושה מושגים שמתערבבים כל הזמן
לפני שנתחיל לתעדף, נעשה סדר במושגים

מושג ראשון: פרומפט

פרומפט הוא הוראה שאנחנו נותנים למודל שפה כדי שיבצע **משימה אחת**. מכניסים קלט, מקבלים פלט.

מדויק

מגדירים בדיוק מה רוצים לקבל

מובנה

בנוי לפי פורמט ברור שהמודל יכול לעבד

חד-פעמי

מופעל, מבצע, מסיים.

נלמד פורמטים של פרומפטים שנכתבו מראש בקפידה, ונפעיל אותם על הפרויקט שלנו.

Follow this instruction carefully.

מושג שני: Agent (סוכן)

סוכן מקבל מטרה ופועל בעצמו כדי להשיגה.

הוא לא מחכה להוראות שלב אחר שלב אלא מקבל החלטות עצמאיות.

בניית סוכן אמיתי היא נושא עומק שלם, שנלמד בהמשך הקורס.

שימוש בכלים 

מפעיל APIs, קבצים,
מסדי נתונים

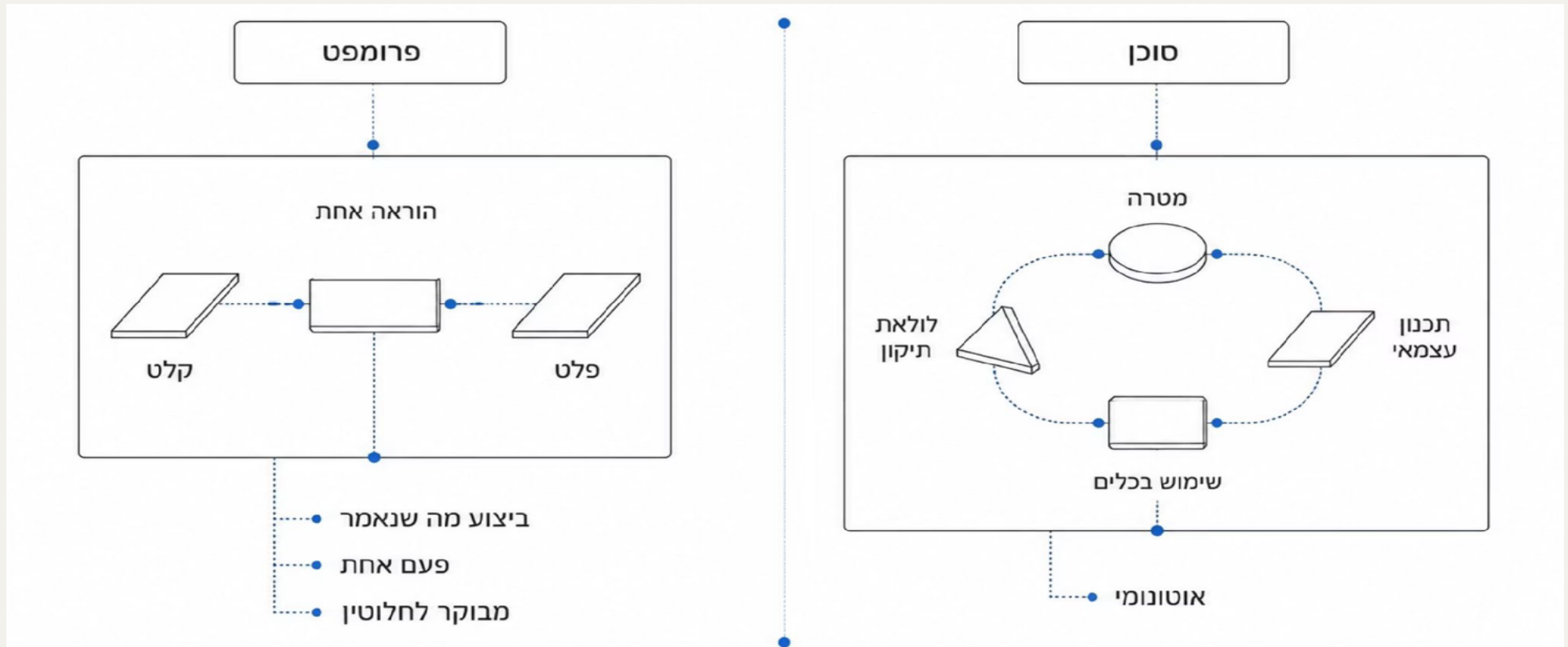
אוטונומיה 

מחליט בעצמו מה
הצעד הבא

לולאה חוזרת 

בודק תוצאה ומתקן את עצמו

פרומפט מול סוכן: ההבדל המהותי



אותו מודל שפה עומד מאחורי שניהם, אבל רמת העצמאות שונה.

פרומפט מבצע מה שאמרנו לו ועוצר.

סוכן משיג מטרה שהצבנו לו, בכוחות עצמו.



Vibe Coding

דרך עבודה, לא כלי

Vibe Coding - גישה שבה אנחנו **מתארים במילים מה אנחנו רוצים שייבנה**, וה-AI כותב את הקוד. האדם מכוון, המודל מבצע.

- מתארים מסבירים ל AI את הכוונה והמטרה
- רואים תוצאה ה-AI מייצר קוד ראשוני/ מערכת ראשונית/ תוצר
- מתקנים בשיחה מכוונים, מדייקים, משפרים
- חוזר חלילה איטרציה מהירה עד לתוצאה הנכונה

שתי משמעויות של בניית מערכות עם "AI"

AI כרכיב שרץ בתוך המערכת

חלק מיכולות המערכת. משתמשים בו כשצריך:

- לנתח
- לסווג
- להסיק מסקנות

AI ככלי בנייה

בניית המערכת או היכולות עם ה-AI

- מתארים במילים
- המודל מייצר קוד
- בונים את כל שכבות המערכת באמצעותו

יכולת יכולה להיבנות באמצעות AI ובכל זאת לא להריץ AI בזמן ההפעלה

מה זה Vibe Coding

✓ Vibe Coding כן

- מאפשר לבנות מהר ולהתנסות
- מוריד את חסם הידע הטכני
- מאיץ את הדרך לתוצאה עובדת
- מאפשר פרוטוטיפינג מהיר

✗ Vibe Coding לא

- לא פוטר אותנו מהבנה של מה בונים
- לא מחליף את שלב האפיון
- לא מבטיח קוד נכון בלי הגדרה ברורה
- לא בא לפני הגדרת הבעיה

בלי הגדרה ברורה של מה בונים ולמה, התוצאה היא קוד שנראה נכון אבל לא נותן מענה לצורך או לרצון.
Vibe Coding בא תמיד **אחרי** האפיון, לא במקומו.

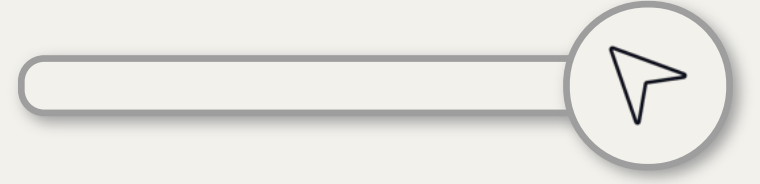
איך שלושת המושגים מתחברים



Agent - מערכת עצמאית
סוכן שפועל בעצמו בתוך
הפתרון שיצרנו



Vibe Coding - בניית הפתרון
עובדים בשיטת Vibe Coding
כדי לבנות את הפתרון המלא
בשיחה עם ה-AI



פרומפט: משימה ממוקדת
כותבים פרומפט לביצוע
משימה ספציפית, כמו
תעדוף רשימת פיצ'רים



מה אנחנו לומדים עכשיו

בשיעור הזה נשתמש בשני פרומפטים בפרויקט שלנו

1. תעדוף

להחליט מה נכנס לגרסה הראשונה ומה ממתין.
לפי ערך מול מאמץ

2. ניתוח תלויות

להבין מה חייב להיבנות לפני מה.
מהו המסלול הקריטי?
אילו פיצ'רים חוסמים אחרים?
מה ניתן לבנות במקביל?

סיכום: שלושה מושגים



Agent

מערכת עצמאית שמקבלת מטרה ומשיגה אותה.
נושא מעמיק למודול הבא בקורס.



Vibe Coding

שיטת עבודה בשיחה עם ה-AI.
האדם מכוון, המודל בונה.
תמיד אחרי האפיון.



פרומפט

הנחיה ברורה - קלט
תשובה- ופלט, ביצוע מדויק וחד-פעמי.
הכלי הבסיסי שמשתמשים בו



תעדוף בעידן ה-AI: הבנת המבנה הארגוני

כשהמאמץ כבר אינו החסם, השאלה היא לא "מה זול
לבנות" אלא "מה כדאי לבנות קודם"

נתחיל: ניצור תחילה את הדרישות הגולמיות לפרויקטים שלנו



פרומפט ליצירת רשימת דרישות גולמית

אתה אנליסט מוצר המתמחה בגזירת דרישות מתוך חומרי אפיון. תפקידך ליצור מכל מה שחקרנו עד כה רשימת דרישות גולמית. חל איסור להמציא דרישות שאין להן עיגון בחומרים.

התייחס לחומרי הפרויקט: [הגדרת הבעיה, מחקר השוק, תחקיר השורש, ומסמך המדדים]. [הפנו למסמכי הפרויקט שלכם].

צור רשימת דרישות גולמית לפי הכללים הבאים:

- כל דרישה היא יכולת בודדת שניתן לתעדף בנפרד.
- כל דרישה חייבת לנבוע משורש בעיה או מפתרון שמופיע בחומר.
- לכל דרישה ציין את המקור, כלומר איזו בעיה או ממצא היא פותרת. אל תוסיף דרישות שאין להן מקור.
- הפרד דרישות פונקציונליות לדרישות של חווית משתמש ומדדי הצלחה, סמן לכל דרישה לאיזה סוג היא שייכת.
- צור טבלה של דרישות, בלי תעדוף ובלי המלצה מה לבנות. התעדוף יתבצע בשלב נפרד.



עכשיו תורכם

צרו את הדרישות של הפרויקט שלכם



המהפך בשאלת התעדוף

בעולם הישן

מאמץ הפיתוח היה החסם היקר.
כל מסגרות התעדוף נועדו לענות:

- על מה כדאי לבזבז שבועות של פיתוח

בעידן ה-AI

מחיר פיתוח המערכת צנח. אב-טיפוס עובד נבנה בשעות. ברגע שהמאמץ מפסיק להיות החסם, הסכנה מתהפכת:

- שהכול קל לבנות, בונים יותר מדי
- מתעלמים משורש הבעיה
- יוצרים פיצ'רים שאינם משרתים אותו.



התעדוף עובר מחיסכון במאמץ להגנה על המיקוד.

אנחנו נכנסים לארגון קיים לא לשדה ריק

מערכת הליבה היא נכס רגיש.

שום ארגון לא יאשר התממשקות אליה
לפני שראה שהפתרון עובד, יציב
ובטוח.

החיבור לליבה הוא איטי מחייב
הרשאות, הוא חסום עקרונית עד
שהמערכת מוכיחה את עצמה.

ההוכחה חייבת לבוא ראשונה, ובנפרד
מהליבה.

אנחנו נכנסים לארגונים
שכבר פועלים.

אנחנו לא בונים סטרט אפ.

ל○ארגון יש מערכות ליבה

ל○המערכות שמריצות את
הפעילות החיונית שלו

ל○מערכת הדיווחים היא
אחת מהן.



הציר המרכזי: שלוש קטגוריות לפי תלות ארגונית

1

עומדת בזכות עצמה

- פועלת על נתונים קיימים.
- נבנית ב-AI אך רצה בלי AI
- זולה לפיתוח ויישום
- תוצאה צפויה
- אין תלות חיצונית
- זה מה שבונים ראשון כדי לבדוק
- אם הקונספט עובד

2

מחייבת AI כחלק ממנה

- הליבה החכמה שמנתחת
- המודל רץ בשוטף
- ערך גבוה
- עלות יכולה להיות גבוהה
- אי-ודאות

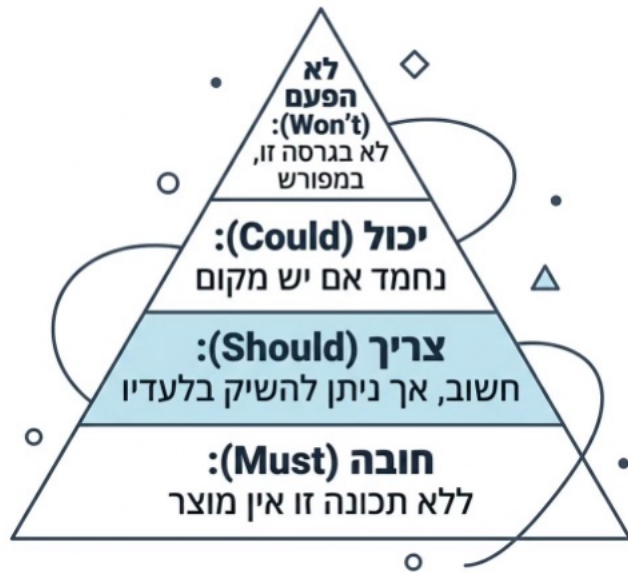
3

מחייבת חיבור למערכות ארגוניות

- תלויה בגישה להרשאות
- אינטגרציה גם מערכות
- החלק האיטי, מסוכן, תלוי
- באחרים.
- נכנס אחרון.

MoSCoW: מה חובה, מה לא עכשיו

היישום על הציר



מה שעומד בזכות עצמו וחשוב למשתמשים נכנס ל-Must ולגרסה הראשונה הוא זה שבודק את הקונספט.

מה שתלוי בחיבור ארגוני נדחה ל-Won't-now, לא מפני שאינו חשוב, אלא מפני שאסור לתלות בו את הוכחת הרעיון.

Must

בלי זה אין מוצר

Should

חשוב, אך אפשר להשיק בלעדיו

Could

נחמד, נכנס אם יש מקום

Won't

לא בגרסה הזו, במפורש

RICE : כשצריך לדרג באותה קטגוריה

Reach

כמה משתמשים מושפעים

Impact

עוצמת ההשפעה

Confidence

כמה אנחנו בטוחים

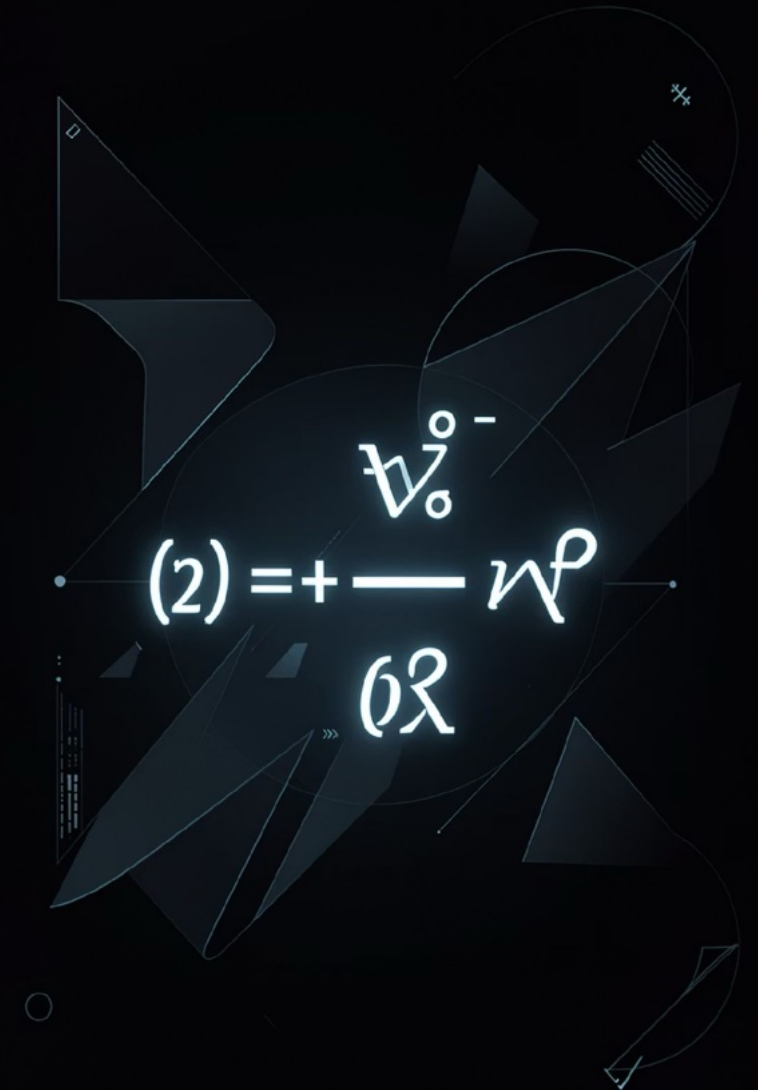
Effort

המאמץ הנדרש

$$\text{ציון} = \text{Effort} \div (\text{Reach} \times \text{Impact} \times \text{Confidence})$$

בעידן ה-AI:

כש Effort הפך לזמין ומיידי, ה- Confidence הופך לבחינה האמיתית. עד כמה אנחנו בטוחים שהיכולת נותנת מענה לשורש הבעיה ולמשתמשים. פיצ'ר קל לבנייה עם ביטחון נמוך לא זוכה לעדיפות.



מטריצת ערך מול מאמץ: הדרך החדשה



מה נפתח ראשון?

ערך נמוך ומאמץ נמוך. האם לפתח?
"למה לא, זה קל וזול". בעידן ה-AI זו מלכודת.

○ קל לבנות

○ מפזר את המיקוד.

ערך נמדד רק מול המהות, המטרה, בעלי העניין והמזדים
שכבר הגדרנו.

← עשה מיד

ערך גבוה, מאמץ נמוך

← תכנן

ערך גבוה, מאמץ גבוה

← הימנע

ערך נמוך, מאמץ גבוה

ערך נמוך, מאמץ נמוך

יישום על מערכת דיווח התקלות מה דעתכם?

העובדים אינם מדווחים על תקלות מפני שמופעלים משני סוגי פחד: חשש מפגיעה אישית וחרדת החלפה. שורש הבעיה המשותף: המערכת אוספת מידע מהעובד ולא מחזירה לו ערך.



תעדוף MoSCoW ותכנון גרסאות

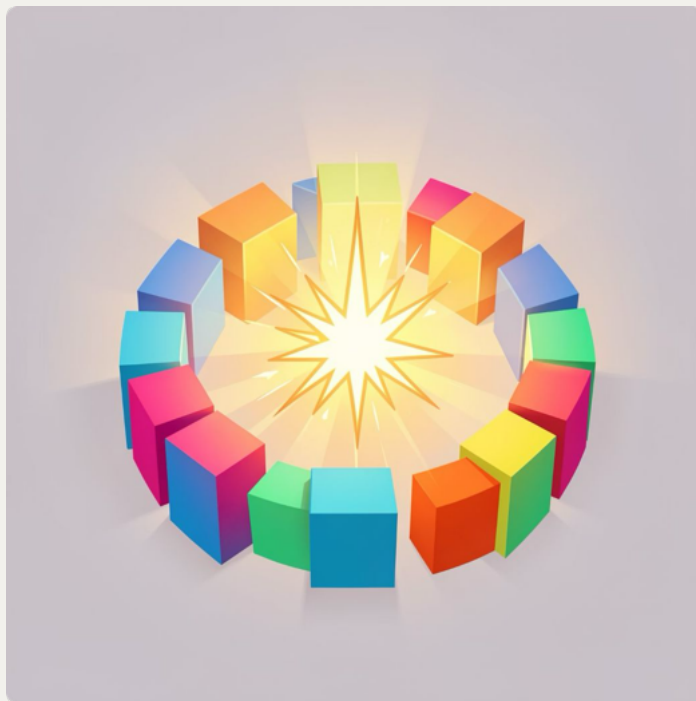
אני מתעדף את המשימות לקראת בניית אב טיפוס.
להלן רשימת המשימות והאילוצים, ממסמכי הפרויקט: [הדביקו או
הפנו לרשימת הדרישות, האילוצים ומסמכים רלוונטיים].

צור תעדוף לפי השלבים הבאים. עצור בסוף כל שלב והמתן לאישורי:

- חלק את המשימות לארבע קטגוריות MoSCoW: חובה, רצוי, אפשרי, ולא עכשיו.
- אחרי שאשר, חשב את זמן הבנייה של משימות החובה והצג כמה אפשרויות.
- אחרי שאשר את תכולת אב הטיפוס, הצע תוכנית לשלוש גרסאות עוקבות.
- בסיום, רכז את כל מה שאישרתי לתוכנית עבודה אחת.



עידן ה-AI

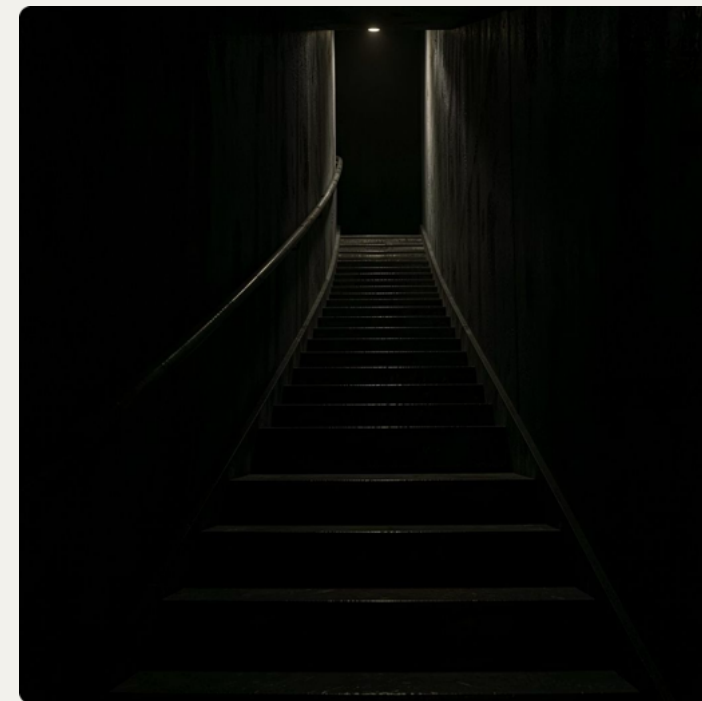


התפיסה החדשה

בונים ראשון את הערך למשתמש.

זהו POC - הוכחת היתכנות של הרעיון, לא של הטכנולוגיה.

האם עובד מוכן לדווח כשהוא רואה שהדיווח מחזיר לו ערך?



התפיסה הישנה

קליטה ← שמירה ← ניתוח ← ערך.

הערך הגיע בסוף, אחרי חודשים עבודה

צריך לבנות תשתית קודם.



בודקים את הרעיון בלי לגעת בליבה, ומארגנים את הממשק מראש

שורש הבעיה שאבחנו:
העובד תופס את הדיווח כפעולה שמסכנת אותו ולא מחזירה לו
דבר. המערכת לא נותנת לו בטחון.

ה-POC- הוכחת יכולת:

העובד מדווח, מקבל משוב חיובי ורואה תגובה גלויה שמישהו טיפל. ☐

בשלב זה בודקים רק את זה. היחיד שנתן מענה לשורש הבעיה ☐

בודקים אותו מבלי להתחבר למערכות הליבה. ☐



בודקים את הרעיון בלי לגעת בליבה, ומארגנים את הממשק מראש

כשיש מערכת אחרת לניהול תקלות
מכיוון שאין גישה לדיווחים אמיתיים
יוצרים דיווחים לדוגמה מקובץ אקסל.

את הפורמט אנחנו לא ממציאים:

בודקים את פורמט מערכת הדיווחים הקיימת

בונים את הקלט על בסיסו.

יוצרים כבר עכשיו מיפוי נכון בין השדות של המערכת שלנו
לשדות של מערכת המקור
שנגיע לחיבור לליבה, אחרי שהכול עובד, המבנה כבר תואם
וההמרה תהיה ממופה.



אחרי שהרעיון אהוב- מרחיבים

רק אחרי שה-POC הוכיח שהעובדים אוהבים את
הפתרון ומדווחים, בונים את יתר השכבות
לפי סדר בתלות הארגונית

הפער שמפיל פרויקטים

מי שמבין את הפער הזה בונה
נכון:
מוכיח ערך בנפרד מהליבה
רק אז מתממשק אליה.

הטעות
להתייחס לזהירות כאיטיות או
כחוסר הבנה.
הארגון מגן על מערכות הליבה
שמריצות את פעילותו, ולא ייתן
לאיש לגעת בהן לפני שהוכח
שהכול עובד ובטוח.
הקלות לייצר מערכת אינה
הקלות להכניס אותה לארגון.

הקלות שבה מייצרים היום
מערכת מפתה.
אפשר לבנות פתרון עובד
בשעה, וקל להיסחף.
מיישמים רבים נכנסים לסחרור
של בנייה, ואז נתקלים בקיר:
הארגון לא מתלהב

מה לא לבנות: ההחלטה הקריטית

ה-AI נוטה להציע עשרה פיצ'רים מפני שהכול אפשרי.
לכל פיצ'ר שואלים: כיצד הוא משרת את בעיית השורש, את בעלי
העניין, את המדדים?

פיצ'ר בלי חיבור

זהו בזבוז

גם אם בנייתו זולה
ומהירה.

פיצ'ר שמשרת

זהו ערך

מחובר לשורש, לבעלי
העניין ולמדדים

בעידן ה-AI המיומנות היקרה אינה לבנות, אלא לדעת מה לא
לבנות למרות שזה אפשרי.



סיכום

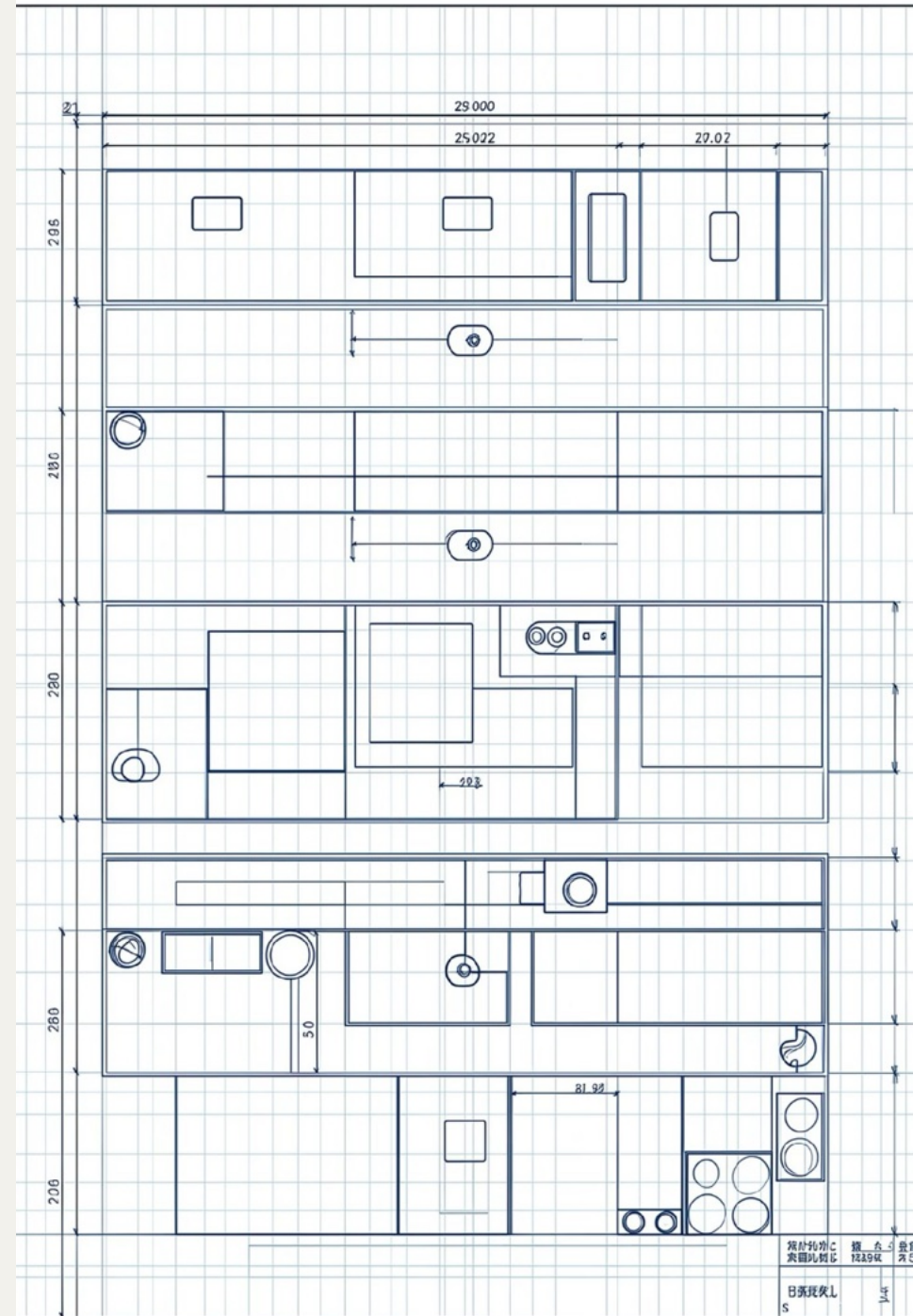
תעדוף הוא לא חיסכון במאמץ
אלא הבנה של המהות הארגונית.



שלוש הקטגוריות
בנייה של המערכת, הוספת AI שמנתח, הוספת
חיבור ארגוני.



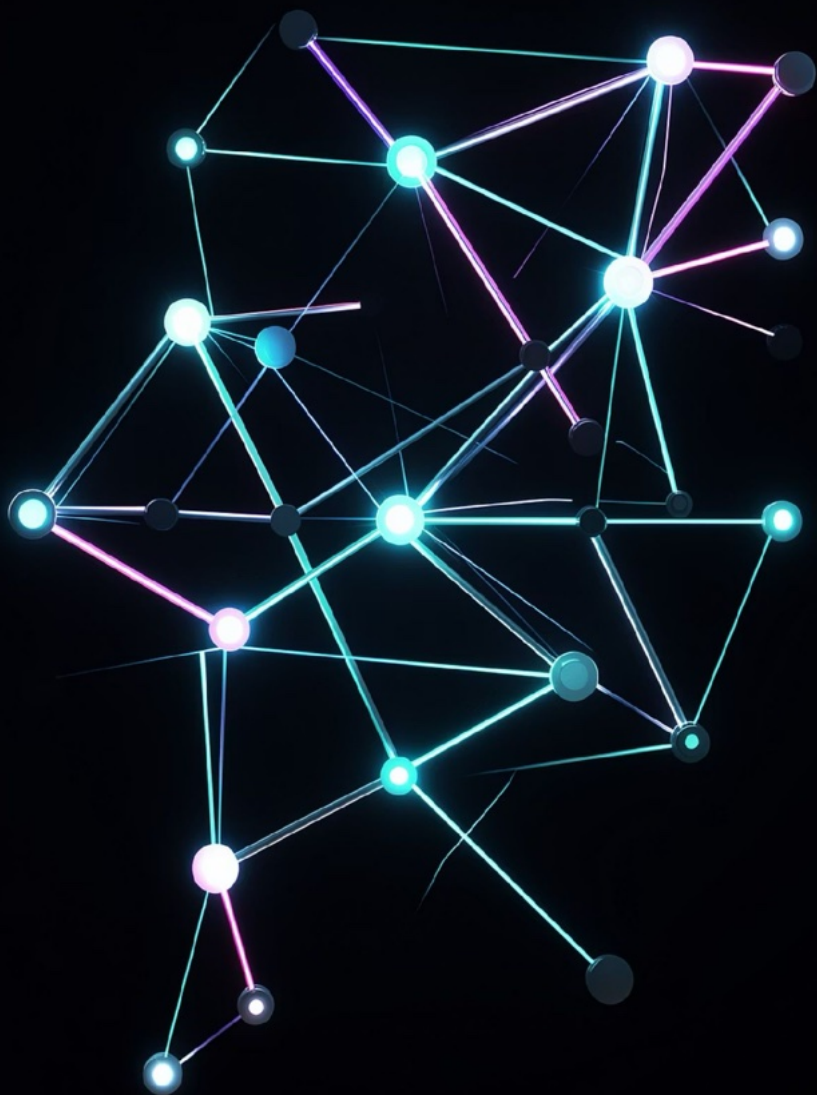
בנייה מודולרית
בונים תחילה את מה שעומד בזכות עצמו
מוכיחים קונספט לפני שממשיכים
בונים מודולרית, כדי לצמוח בלי לשבור.



עכשיו תורכם

צרו את התעדוף לדרישות שיצרתם





תלויות בעידן ה-AI: הנתיב הקריטי הוא הארגון

PERT-ו CPM

- קביעת סדר משימות
- זיהוי מה חוסם את הוכחת הרעיון

מה זו תלות

תלות היא יחס "לפני-אחרי": משימה שלא יכולה להתחיל לפני שאחרת הסתיימה.

בעידן ה-AI נוסף סוג תלות חדש- תלות לוגית:

- תלות בזרימת מידע בין רכיבים
- יכולת שתלויה בכך שמאגר נתונים קיים
- שסוכן קיבל פלט מסוכן אחר.

תלות לוגית (AI)

זרימת מידע:

סוכן מחכה לפלט מסוכן אחר
מודל מחכה לנתונים שהוזנו

תלות קלאסית

סדר ביצוע:

משימה ב' מתחילה רק
אחרי שמשימה א'
הסתיימה



דוגמה- הכנת פסטה

בואו ונכין פסטה:

- נרתיח מים (10 דק')
- נחתוך ירקות (5 דק')
- נבשל רוטב (15 דק')
- נבשל פסטה (9 דק' אחרי שהמים רתחו),
- נסנן נערבב עם הרוטב ונצלחת (3 דקות)
- נגיש.

ניתן לבצע חלק מהמשימות במקביל

- חותכים ירקות בזמן שהמים מתחממים.

בישול הפסטה (כולל הרתחת המים, הבישול והצילחות
 $10+9+2$ (21 דק')

זוהי השרשרת הארוכה ביותר שאי אפשר לקצר או לבצע במקביל.

הנתיב הקריטי: רצף המשימות הארוך ביותר שקובע את זמן הסיום הכולל. עיכוב בו מעכב את הכול.





שאלה

מה אם נצליח לקצר את משך הרתחת המים ל 3 דקות?
מה יהיה הנתביב הקריטי?

פרומפט לניתוח תלויות וסדר ביצוע

אתה מומחה לניהול פרויקטים ומתמחה בניתוח תלויות ובתכנון סדר הביצוע.

תפקידך לקבל רשימת משימות ולקבוע באיזה סדר נכון לבנות אותן.

התייחס לרשימת הדרישות של הפרויקט. קרא בנוסף את כל מסמכי הפרויקט. [הפנו לרשימת הדרישות].

נתח את התלויות ביניהן והחזר שלושה דברים:

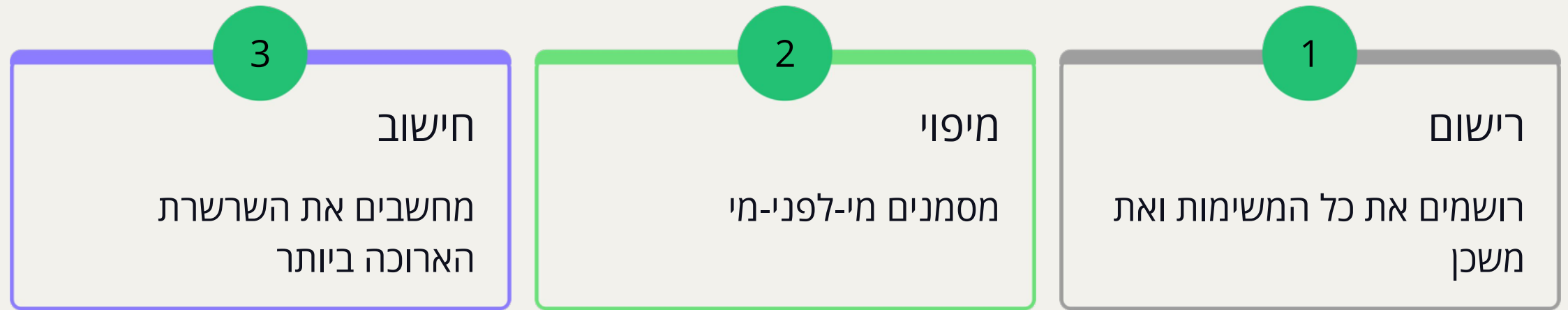
- טבלה שבה לכל משימה רשום במה היא תלויה ולמה.
- תרשים זרימה פשוט בחצים שמראה את סדר הביצוע ומה אפשר לעשות במקביל.
- זיהוי הנתיב הקריטי, הרצף הארוך ביותר שקובע את משך הפרויקט.

עבוד שלב אחרי שלב, ועצור לאישורי לפני שתמשיך לשלב הבא.



CPM: שיטת הנתיב הקריטי

CPM מתאים כשזמני המשימות ידועים ויציבים.



מחוץ לנתיב הקריטי

משימות "צפות": שניתן לבצע במקביל בלי לפגוע בסיום הכולל.

בנתיב הקריטי

הקפדה יתרה. כל עיכוב בו מתגלגל ישירות לסיום הפרויקט.

PERT: במצב של חוסר ודאות

כשלא יודעים המשך המדויק, PERT מעריך: אומדן אופטימי (O), אומדן סביר (M), אומדן פסימי (P).

$$E = \frac{O + 4M + P}{6}$$

ה- E - המשך שכנראה יקרה. מספר יחיד שמתייחס לאי-הוודאות במקום להעמיד פנים שהיא לא קיימת.

O — אופטימי

כשהכול מסתדר בצורה הטובה ביותר האפשרית

M — סביר

ההערכה הריאלית הסבירה ביותר. מקבל משקל פי ארבע

P — פסימי

התרחיש הגרוע: עיכובים, אישורים, בעיות בלתי צפויות

בעידן ה-AI אי-הוודאות הגדולה נמצאת דווקא בהבנה ומשימות החיבור הארגוני - שם PERT הכרחי.

קשר לתעדוף: מה חוסם את הוכחת היכולת POC

ממיינים את ה-MUST לשלוש קטגוריות ושואלים: איזו מהן נמצאת על הנתיב הקריטי?

מחייב חיבור ארגוני
ארוך, לא ודאי, תלוי
באישורים של אחרים.

זה הנתיב הקריטי
האמיתי. מזהים אותו
ומתכננים סביבו.

מחייב AI שמנתח
תלוי בכך שיש קלט
לעבד.

מגיע אחרי שהשכבה
הראשונה פועלת.
עולה כסף (עיבוד)

עומד בזכות עצמו
קצר, ודאי, קל ליישום
ובעל ערך.

מתחילים בו. הוכחת ערך
מיידית.

יישום על דיווח התקלות

הנתיב הקריטי הוא ארגוני:

הרשאות גישה למערכות הליבה של הארגון והאינטגרציה איתן.
המשימה הארוכה, החוסמת, תלויה באישורים של אבטחת מידע.

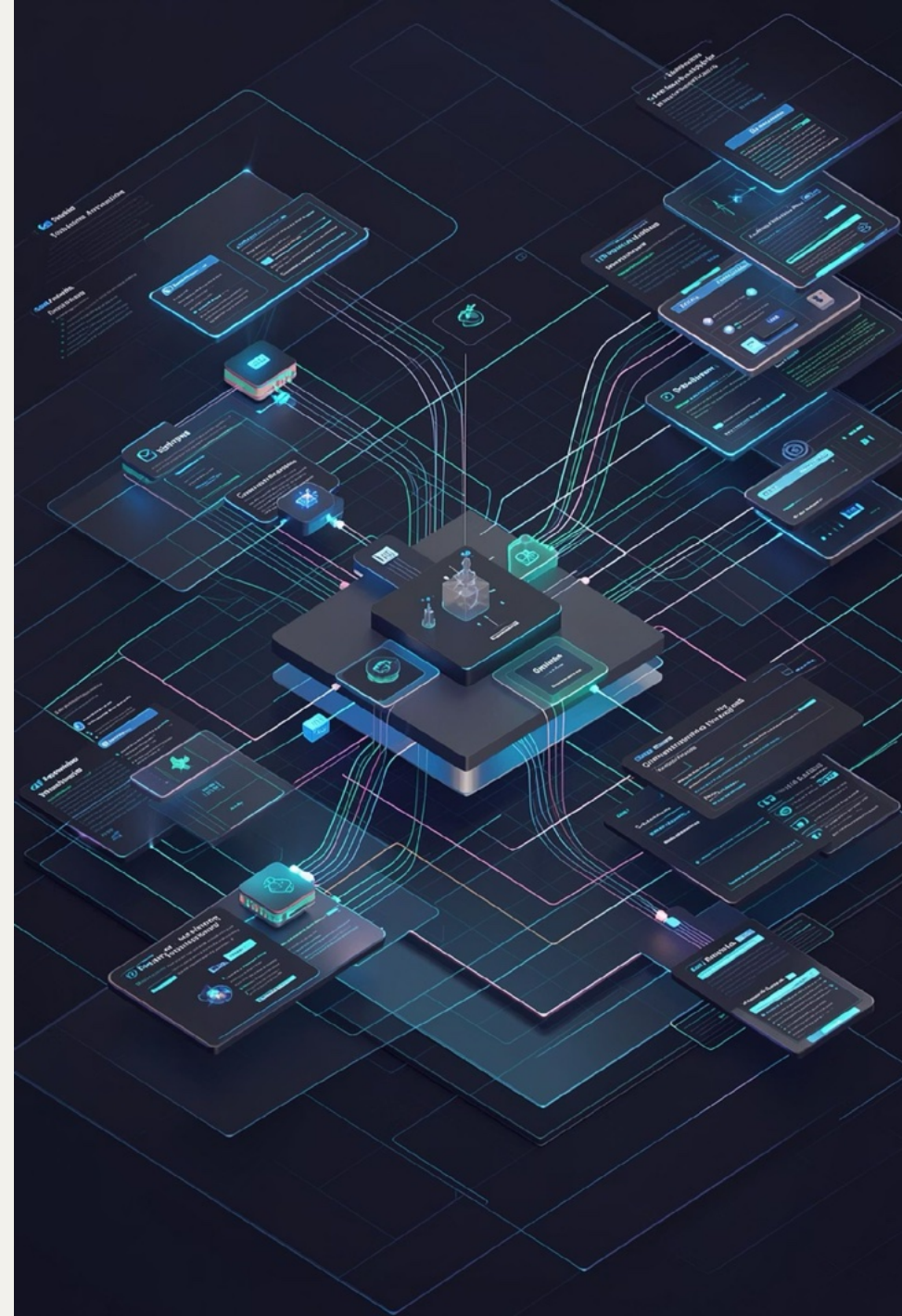
מה מוכיחים

שהעובדים מדווחים
כשהמערכת פשוטה ומקבלים
חיווי חיובי מבלי להיחשף
לפני שנוגעים בחיבור
למערכות הארגון.

מה בונים תחילה

קליטת דיווח ממסמך, שיוך
פנימי, שמירה והצגה
שכבה שנבנית ב-AI,
רצה בלי AI, ואינה תלויה
בשום מערכת חיצונית.

בונים מה שאינו על הנתיב הקריטי תחילה: מוכיחים ערך
בזמן שהאישורים הארגוניים מתקדמים ברקע.



PERT על דיווח התקלות

משימת "חיבור למערכת המשאבים הארגונית":

15

ימים - סביר

תהליך רגיל עם מספר סבבי אישור

5

ימים - אופטימי

ההרשאות מאושרות מיד, ללא עיכובים בירוקרטיים

17.5

ימים - הערכת PERT

$$17.5 = 6 \div (40 + 15 \times 4 + 5)$$

40

ימים - פסימי

ועדת אבטחת מידע, סבבי אישורים מרובים

$$E = \frac{5 + 4 \times 15 + 40}{6} = \frac{105}{6} = 17.5 \text{ ימים}$$

הטווח מציג את הסיכונים ולכן חושף למה אסור לתלות בו את הוכחת הקונספט.

עכשיו תורכם

הוסיפו את התלויות של הפרויקט שלכם



מ-AI לגרף — נכיר את Mermaid

בקשו מה-AI לצייר את הנתיב הקריטי בפורמט MERMAID. תרשים לדוגמה:

```
```mermaid
graph LR
 A["שיוך ושמירה - 2 ימים"] --> B["קליטת דיווח - 3 ימים"]
 B --> C["תצוגה למדווח - 3 ימים"]
 A --> D["AI ניתוח סנטימנט - 8 ימים"]
 D --> E["חיבור ארגוני - 17 ימים"]
 E --> F["התראות למנהלים - 4 ימים"]
 C --> G["סגירת מעגל ערך"]
 F --> G
```
```

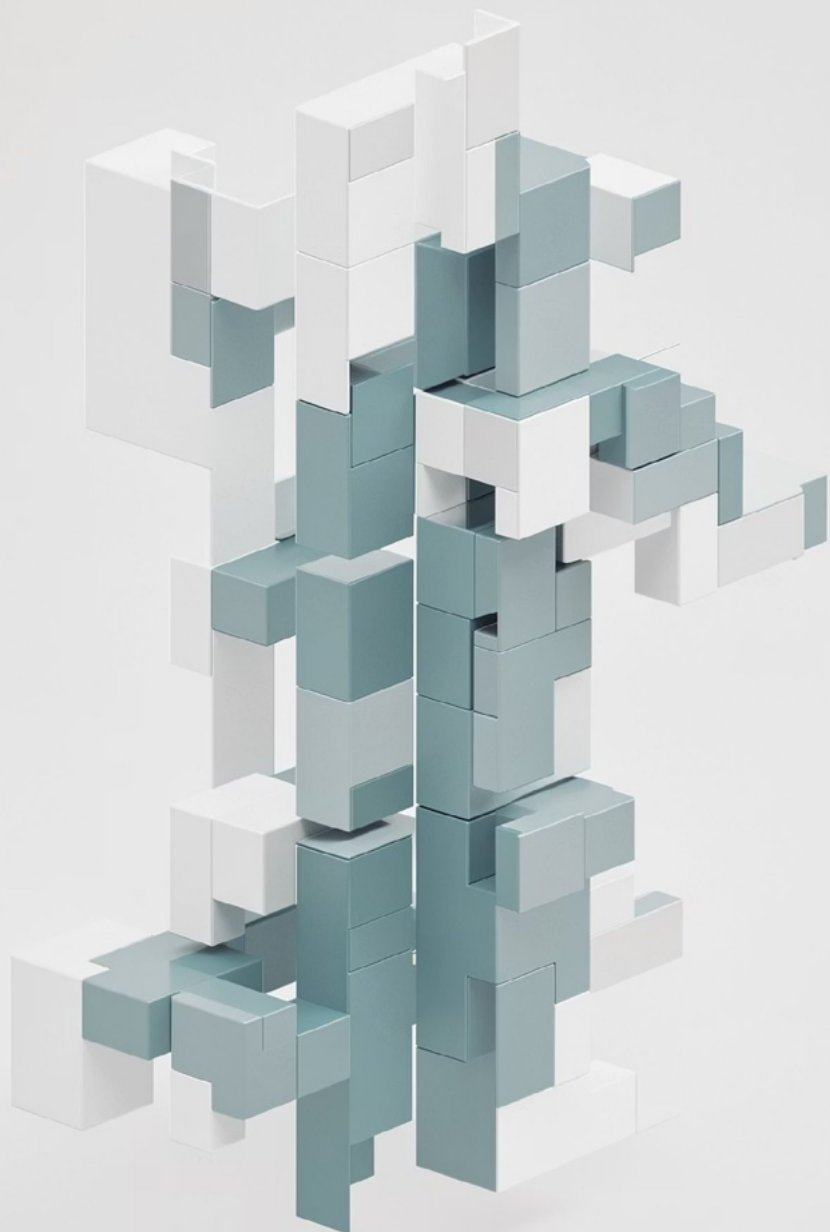
נריץ את הקוד ב DRAWIO כדי שנוכל לערוך

צרו את תרשימים MERMAID לתלויות של הפרויקט שלכם





המסלול הקריטי (32 ימים) ארוך פי שניים מהמסלול המהיר (15 ימים). לכן בונים את המסלול המהיר תחילה ומוכיחים
ערך בזמן שהחיבור הארגוני מתקדם.



סיכום

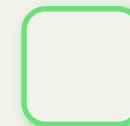
תלויות הן זיהוי חסמים - מה חוסם את הוכחת הרעיון.



PERT חישוב בחוסר ודאות

CPM כשהזמן ודאי

הנתיב הקריטי בפרויקטי AI בארגון הוא לרוב החיבור למערכות הליבה.



בנייה מודולרית

כל מדול וכל שכבה עצמאיים, מתחברים דרך חוזה קבוע, לכן אפשר לצמוח בלי לשבור את מה שכבר עובד.



רטרוספקטיבה

שלוש שאלות לסיום

1

מה למדתי היום שלא ידעתי?

תובנה חדשה שמשנה את
האופן שבו אני חושב על ניהול
מוצר.

2

מה הפתיע אותי?

משהו שלא ציפיתי לו - תפיסה
שהתהפכה, הנחה שנשברה.

3

מה אקח איתי כבר מחר?

פעולה קונקרטית אחת שאוכל
ליישם מחר בבוקר בעבודה.



תודה רבה- זכרו

“

עיקרון ראשון
"ככל שקל יותר לבנות, ההחלטה האם בונים
הופכת קריטית יותר."

”

“

עיקרון שני
"בודקים את כל העלויות ולא רק את עלויות הפיתוח"

”

תודה רבה

רילי לבבי

053-4365200

levavir@gmail.com

